9.2 子代理与跨Agent通信

> **生成模型**：GPT-5.4 (openai/gpt-5.4)
> **Token 消耗**：输入 ~35k tokens，输出 ~4k tokens（估算，本节）

上一节讲的是“静态分工”：哪些消息先路由给哪个 Agent。这一节往前走一步，讲“动态协作”：一个 Agent 正在干活时，怎么把一部分任务外包给另一个 Agent，干完之后再把结果拿回来继续做。

这就是子代理和跨 Agent 通信的价值。它解决的不是“入口分流”，而是“执行过程中如何分工”。比如 code Agent 正在实现一个功能，但 API 文档细节不确定；这时最稳的做法，不是它自己一边写代码一边开十几个网页乱查，而是拉起一个 research 子代理，让对方只负责查证，然后把结论发回来。

> **衍生解释：子代理**
>
> 子代理可以理解成“临时拉来的专员”。主 Agent 负责总任务，子代理只拿一个明确子任务，干完就回报。这样做的好处，是不同任务的上下文不会糊成一锅粥。

9.2.1 `sessions_spawn` ：把子任务派出去

OpenClaw 里，派生子代理最直接的工具是 `sessions_spawn` 。它的典型参数很简单：任务描述 `task` ，可选的目标 `agentId` ，可选模型覆盖 `model` ，以及便于识别的 `label` 。

这几个参数背后，其实对应着很实用的决策：

- `task` ：把任务说清楚，别把总目标一股脑塞给子代理。
- `agentId` ：决定叫谁来干。
- `label` ：方便主 Agent 后面按标签找到这个会话。
- `cleanup` ：决定干完后删不删记录。

一个典型场景是这样：

1. `code` 收到“给项目接一个新支付 SDK”。
2. 它先拆任务，发现文档和版本差异不确定。
3. 它调用 `sessions_spawn` 拉起 `research` 。
4. `research` 只查 SDK 文档、错误码、签名要求。
5. `code` 拿到结论后继续改代码和补测试。

这里最重要的不是“派出去”这一下，而是任务切分的粒度。子代理不是让你把整个任务甩锅出去，而是把适合独立完成的一段工作抽出来。

9.2.2 会话隔离：为什么子代理越独立越好

很多人第一次看到子代理，会自然地以为“那是不是共享同一份上下文更方便”。恰恰相反，OpenClaw 里一个很重要的设计就是：主代理和子代理不是同一个会话空间。

这点特别关键。研究型任务会产生大量网页摘要、术语、版本差异、广告废话。如果这些东西直接冲进 code Agent 的实现上下文，模型很容易开始绕、开始发散，甚至忘了当前到底要改哪几个文件。

会话隔离的好处有三个。

第一，主任务上下文更干净。主 Agent 只拿到“已整理的结果”，而不是全部中间噪音。

第二，权限边界更稳。比如 research 只给浏览器和读取文档权限，不给写代码；code 则反过来。

第三，更容易复查。你后面可以单独打开某个子代理会话，看它到底查过什么、得出了什么结论。

9.2.3 `sessions_send`：跨 Agent 传话的主干道

如果说 `sessions_spawn` 像“创建一个专员”，那 `sessions_send` 更像“给另一个会话发工作消息”。它是跨 Agent 通信里最常用的动作。

它至少能干三类事。

第一类，给已有子代理补充任务。比如 research 已经在跑，你又想让它顺手比较一下旧版 API 和新版 API 的差别。

第二类，把整理过的结果发回主 Agent。比如 deploy Agent 部署完成后，用 `sessions_send` 把版本号、回滚点、日志摘要发给 code review Agent。

第三类，在两个专职 Agent 之间建立配合。比如 customer service Agent 先把用户问题归类，再把高风险工单发给 escalation Agent。

一个常见配法如下：

```
{
  agents: {
    list: [
      {
        id: "code",
        model: "anthropic/claude-sonnet-4-5",
        skills: ["coding-agent", "github"],
        tools: {
          allow: ["read", "write", "edit", "bash", "sessions_spawn", "sessions_list", "
        }
      },
      {
        id: "research",
        model: "google/gemini-2.5-pro",
        skills: ["browser"],
        tools: {
          allow: ["browser", "read", "sessions_send", "sessions_history"],
          deny: ["write", "edit", "bash"]
        }
      }
    ]
  },
  tools: {
    agentToAgent: {
      enabled: true,
      allow: [
        { from: "code", to: "research" },
        { from: "research", to: "code" }
      ]
    }
  }
}
```

这个配置有个很值得记住的地方：跨 Agent 通信最好显式开启、显式放行，而不是默认让所有人都能互相打扰。团队里不是每个人都能随便给所有人派活，Agent 也一样。

9.2.4 `sessions_list` 和 `sessions_history`：找人、看记录

多 Agent 一旦多起来，主 Agent 不一定记得某个子代理会话的精确键值。这时就会先用 `sessions_list` 找会话，再用 `sessions_history` 看最近记录。

你可以把这两个工具理解成：

- `sessions_list` ： 通讯录 + 最近会话列表。
- `sessions_history` ： 翻聊天记录。

实际协作里，这两个工具特别适合做“先看后发”。比如主 Agent 想知道 research 那边查到哪一步了，不一定要马上发消息催。它可以先读历史摘要，确认对方已经查过哪些资料、现在卡在哪，再决定下一步是等待、补充任务，还是直接接手。

这种模式看起来小，但非常实战。因为多 Agent 真正的效率，不只是能发消息，而是能避免无效消息。

9.2.5 Reply Ping-Pong：两个 Agent 来回对话，不是玄学，是控轮次

OpenClaw 里还有一个很有意思的机制，叫 Reply Ping-Pong。简单说，就是 `sessions_send` 发出第一条消息后，两个 Agent 不一定只交流一轮；它们可以围绕同一个问题来回补充几轮，再把最终结果汇总出来。

这件事听上去很炫，但实战里一定要克制。因为一旦不控轮次，就会出现两个 Agent 互相解释、互相重述、越聊越长，最后消耗了很多 token，真正的信息密度却没提升多少。

所以 Reply Ping-Pong 的正确用法，不是“让它们多聊会儿”，而是：

- 只用于需要澄清的任务。
- 明确最大轮次。
- 每轮都围绕一个窄问题。

举个例子。 `writer` Agent 要写一份技术总结，但对 `research` 发来的资料里有两个版本结论不一致。它可以发起一次短 ping-pong：先问差异点，再问最终建议，然后停。这样有用。要是让两个 Agent 就“这段表述怎么更完整”来回打磨五六轮，基本就开始浪费了。

9.2.6 三种最常见的委派模式

模式一：查证型委派

主 Agent 负责主线推进，子代理只负责查资料、比对文档、确认接口。这个是最稳的，也是最推荐优先使用的。因为研究型任务和执行型任务本来就该分开。

模式二：审查型委派

主 Agent 完成第一版结果后，再拉一个子代理做 review。比如 code Agent 写完脚本后，交给 reviewer Agent 做“危险命令检查”或“边界条件补充”。这种模式很像真实团队里的二次审查。

模式三：执行型委派

主 Agent 做计划和判断，子代理负责落地动作。比如 release Agent 决定要发布，deploy Agent 真正执行部署、收日志、回传状态。这个模式对权限边界要求最高，别让所有 Agent 都能执行。

9.2.7 一个完整例子：编码 Agent + 研究 Agent + 写作 Agent

下面这份配置，适合做“先查资料，再改代码，最后生成说明文档”的完整协作链。

```
{
  agents: {
    list: [
      {
        id: "code",
        name: "Code Agent",
        model: "anthropic/claude-sonnet-4-5",
        workspace: "~/projects/app",
        skills: ["coding-agent", "github"],
        tools: {
          allow: ["read", "write", "edit", "bash", "sessions_spawn", "sessions_list", "
        }
      },
      {
        id: "research",
        name: "Research Agent",
        model: "google/gemini-2.5-pro",
        skills: ["browser"],
        tools: {
          allow: ["browser", "read", "sessions_send", "sessions_history"],
          deny: ["write", "edit", "bash"]
        }
      },
      {
        id: "writer",
        name: "Writer Agent",
        model: "openai/gpt-5.2",
        skills: ["memory"],
        tools: {
          allow: ["read", "write", "edit", "sessions_history", "sessions_send"],
          deny: ["bash"]
        }
      }
    ]
  },
  tools: {
    agentToAgent: {
      enabled: true,
      allow: [
        { from: "code", to: "research" },
        { from: "research", to: "code" },
        { from: "code", to: "writer" },
        { from: "writer", to: "code" }
```

```
        ]
      }
    }
  }
```

这条链路很好理解： `code` 是总负责人， `research` 给事实依据， `writer` 输出最终文档。每个人各管一摊，不会互相污染上下文，也不会让“会写文档的 Agent”去直接改仓库。

9.2.8 实战建议：先收窄，再放宽

如果你第一次配子代理，最稳的顺序不是追求复杂协作，而是这样：

1. 先做一个主 Agent + 一个 research 子代理。
2. 只开放 `sessions_spawn` 和必要的 `sessions_send` 。
3. 先跑查证型委派，不急着搞多轮 ping-pong。
4. 稳定之后，再加入 reviewer、writer、deploy 这类角色。

多 Agent 协作最怕的是“结构上很高级，行为上很混乱”。只要你记住一句话就不容易跑偏：主 Agent 负责目标和整合，子代理负责一段明确、可独立完成的子任务。

本节小结

子代理解决的是执行过程中的动态分工。 `sessions_spawn` 用来拉起专门处理子任务的会话， `sessions_send` 用来跨 Agent 传话， `sessions_list` 和 `sessions_history` 则负责找会话、看记录。真正让这套机制稳定的，不是工具名字本身，而是三件事：任务要切得清楚、会话要保持隔离、通信权限要显式收紧。这样一来，多 Agent 才像团队协作，而不是多个模型在同一个系统里互相打断。下一节我们把这些机制放进更真实的场景里，看代码评审、部署、客服、研究写作这些流程怎么落地。